Model Predictive Control for Thermal Optimization: Implementation Resources, Comparative Analyses, and Machine Learning Integration

Theoretical and Practical Resources for Studying MPC Implementation

Developing a robust Model Predictive Control (MPC) system requires a solid grasp of state-space system representation, optimization theory, and time-discretization algorithms.[1] To transition from a telemetry-heavy, model-free paradigm like Reinforcement Learning (RL) to a model-based control architecture, control engineers must consult established textbooks, specialized software frameworks, and practical tutorials.[1] These materials build the necessary mathematical foundation, progressing from basic linear formulations to complex nonlinear and hybrid implementations.[1]

Foundational Textbooks and Academic Literature

Several authoritative texts cover the core principles of predictive control, providing distinct theoretical frameworks and implementation guidelines:

- **Predictive Control for Linear and Hybrid Systems** by Francesco Borrelli, Alberto Bemporad, and Manfred Morari.[4] This text is a comprehensive resource for understanding the stability, recursive feasibility, and robustness of predictive controllers.[4] It provides a rigorous mathematical treatment of explicit MPC, a methodology where the multi-parametric quadratic program is solved offline.[4] The resulting control law is precomputed as a piecewise affine function over a partitioned polyhedral state-space, allowing microcontrollers to evaluate optimal control inputs via a simple lookup table without running real-time optimization solvers.[4] The textbook is supported by practical exercises utilizing the Multi-Parametric Toolbox 3.0 (MPT3) in MATLAB.[4]
- **Predictive Control: Fundamentals and Developments**.[1] This book provides an exhaustive introduction to classical industrial algorithms, trace-level system identification, and parameter tuning.[1] It covers the algorithmic derivation of step-response models, such as Dynamic Matrix Control (DMC), and difference equation models, such as Generalized Predictive Control (GPC).[1] It also bridges the gap between open-loop finite-horizon optimization and closed-loop infinite-horizon stability.[1]
- **Model Predictive Control System Design and Implementation Using MATLAB** by Liuping Wang.[3] Designed for both academic and industrial practitioners, this monograph

focuses on state-space MPC formulations embedded with integrators to eliminate steady-state offsets.[3] The text details continuous-time and discrete-time system formulations and introduces the use of orthonormal basis functions, such as Laguerre and exponential functions, to represent control trajectories.[3] This parameterization reduces the number of decision variables in the online optimization problem, enabling faster solve times on standard industrial automation hardware.[3]

- **Model Predictive Control: Fundamentals and Practice** by Jay H. Lee, Niket S. Kaisare, and Carlos E. Garcia.[1] This text is highly regarded for its practical focus on industrial process control applications, parameter tuning, and the design of state estimators like Kalman Filters to handle unmeasured process disturbances.[1]

Implementation Tutorials and Open-Source Codebases

To bridge the gap between theoretical models and physical deployment, hands-on software tutorials provide a practical entry point.[2] A notable example is Willem-Daniel Esterhuizen's "Model Predictive Control Basics" tutorial.[2] This resource guides developers through formulating and solving a discrete-time, finite-horizon Optimal Control Problem (OCP) using Python.[2] The tutorial uses a double integrator system to demonstrate how state and input constraints are defined and solved using CasADi's opti stack and the interior-point solver IPOPT.[2]

Resource Name	Authors / Sources	Primary Technical Focus	Core Algorithms & Methods	Codebase / Software Requirements
Predictive Control for Linear and Hybrid Systems [4]	F. Borrelli, A. Bemporad, M. Morari	Stability, feasibility, robustness, and explicit/hybrid control systems.[4]	Explicit MPC, multi-parametric programming, polyhedral manipulation.[4]	MATLAB, Multi-Parametric Toolbox (MPT 3.0) [4]
Model Predictive Control System Design and Implementation [3]	Liuping Wang	Industrial process application, state-space design, parameter tuning.[3]	Embedded integrators, Laguerre orthonormal basis functions.[3]	MATLAB, Simulink, custom control libraries [3]
Predictive	Academic	Evolution of	Dynamic	Analytical

Control: Fundamentals and Developments [1]	Monograph	MPC, classical algorithms, parameter design, robust control.[1]	Matrix Control (DMC), Generalized Predictive Control (GPC).[1]	state-space models and difference solvers [1]
Model Predictive Control Basics Tutorial [2]	W. D. Esterhuizen (Towards Data Science)	Practical discrete-time NMPC modeling, optimization, and loop simulation.[2]	Finite-horizon OCP, receding horizon feedback loop, constraints handling.[2]	Python, CasADi, SciPy, NumPy, IPOPT [2]

Comparative Analyses of Model Predictive Control and Reinforcement Learning

The selection of a control paradigm for safety-critical thermal environments, such as data centers and building HVAC loops, requires a rigorous comparison of Model Predictive Control and Reinforcement Learning.[7] These two approaches solve the autonomous optimization problem from opposite directions, resulting in distinct trade-offs in deployment overhead, sample efficiency, safety guarantees, and online computational requirements.[8]

Dynamic Paradigms and Key Distinctions

Model Predictive Control relies on an explicit mathematical model of the process, derived from physical first principles or rigorous system identification.[8] At each step, it solves an online constrained optimization problem over a rolling horizon, applying only the first calculated input before repeating the process.[8] Conversely, Reinforcement Learning is a model-free, data-driven approach where an agent learns an optimal policy through trial-and-error interactions with an environment, maximizing a scalar reward function.[8]
This fundamental difference creates distinct operational profiles:

- **Constraint Handling and Safety:** MPC explicitly incorporates physical, operational, and safety boundaries as hard mathematical constraints directly within its optimization formulation.[2] This ensures that the controller avoids actions that violate equipment limits or thermal safety boundaries.[7] RL, on the other hand, relies on soft penalty terms within its reward function to discourage unsafe states.[2] Consequently, RL can only offer probabilistic safety guarantees, and its trial-and-error exploration phase carries risks of equipment damage or operational instability when deployed on physical systems.[8]
- **Data and Computational Allocation:** The engineering effort for MPC is front-loaded.[8] Substantial domain expertise is required to construct a highly accurate physical or

grey-box model before deployment.[8] Once deployed, however, it requires minimal ongoing data.[8] The online phase is computationally demanding, requiring the iterative solution of numerical optimization problems at every time step.[14] For RL, the computational burden is back-loaded.[14] Training an agent requires millions of interaction steps, typically necessitating high-fidelity simulators to avoid real-world damage.[8] Once the policy is trained, online deployment is highly efficient, requiring only a simple neural network forward pass.[14]

Quantitative Findings from Field Deployments

While simulation studies often show ideal performance for both controllers, physical field trials reveal how real-world disturbances, occupancy behavior, and model-plant mismatches affect system performance.[9]

Legacy Base Controller =>
|
+---> Model Predictive Control (MPC) =>
| * Maintains strict occupant comfort boundaries
| * Superior efficiency when normalized for comfort
|
+---> Reinforcement Learning (Ibex-RL) =>
* Marginally higher raw energy savings
* Introduces significantly higher occupant discomfort

A key direct comparison was documented in a month-long field trial in an occupied home with a heat pump system in West Lafayette, Indiana.[9] This study compared a physics-informed MPC against a model-based RL framework named "Ibex-RL".[9] Ibex-RL automatically learned a physics-informed system dynamics model and optimized a multi-objective reward function using stepped setpoint changes to prevent the activation of auxiliary electric resistance backup heat.[9]
The experimental results showed that both advanced controllers achieved significant energy savings compared to the home's legacy baseline controller.[9] The RL agent achieved a 22% reduction in energy consumption, marginally outperforming the MPC's 20% savings.[9] However, this raw energy saving was accompanied by a trade-off: the RL agent allowed significantly higher occupant thermal discomfort.[9] When the energy savings were normalized to the exact level of thermal comfort provided, MPC demonstrated superior overall operational efficiency, safety, and robustness.[9]
This direct comparison highlighted key challenges in real-world deployments:

- **Initialization Safety:** Safely initializing learning-based controllers in a physical system without causing system failure or immediate discomfort is difficult.[9]

- **Control Mismatch:** Physical HVAC actuators often behave nonlinearly or possess discrete operational modes that do not perfectly map to continuous control outputs, causing model-plant mismatches.[9]
- **Online Learning Stability:** Maintaining the stability and mathematical correctness of an online learning algorithm in a chaotic, live environment with stochastic occupancy and weather events is exceptionally difficult.[9]

These physical challenges are compounded by systemic biases in historical academic literature.[16] A comprehensive review of 104 peer-reviewed advanced HVAC control field demonstrations (24 residential and 80 commercial) revealed that 71% of published studies used experimental evaluation protocols that produced unreliable performance estimates.[16] The primary source of this bias is "overestimation error" in multi-zone buildings.[20] Researchers frequently control only a small subset of thermal zones within a larger facility and calculate energy savings solely based on those zones, ignoring the lateral heat transfer from adjacent, uncontrolled zones.[20] This lateral thermal leakage essentially offloads the heating or cooling load to the adjacent zones, making the perceived localized savings fictitious.[20] When analyzing only the 29% of field studies with reliable, heat-transfer-corrected protocols, the duration-weighted average cost savings were found to be 16% in residential buildings and 13% in commercial buildings.[16]

In high-density thermal environments, such as modern data centers, thermal management is directly tied to the Power Usage Effectiveness (PUE) metric.[7] PUE is defined as the ratio of total facility energy consumption to IT equipment energy consumption.[12] With typical data center PUE values ranging from 1.3 to over 2.0, a significant portion of total operational energy is consumed by cooling loops, computer room air-handling (CRAH) units, and chiller plants.[12] Deployed in this context, MPC utilizes multivariable predictive capabilities to coordinate tightly coupled variables—including airflow, water loops, humidity, and temperatures—proactively rather than reactively.[7] By forecasting future server compute loads and ambient weather conditions, MPC prevents "over-cooling" and reduces transient thermal oscillations, driving PUE closer to the ideal limit of 1.0.[7]

Research Paper Title	Authors & Year	Experimental Context & Location	Key Algorithms Evaluated	Core Findings & Normalized Savings
Comparative Field Deployment of Reinforcement Learning and Model	Mulayim et al., 2025	Occupied home with heat pump, West Lafayette, Indiana.[9]	Physics-informed MPC vs. Ibex-RL.[9]	RL achieved 22% raw savings; MPC achieved 20% savings but demonstrated superior

Predictive Control [9]				comfort-normalized efficiency.[9]
Lessons learned from field demonstrations of MPC and RL for residential and commercial HVAC [16]	Khabbazi et al., 2025	Comprehensive review of 104 papers.[16]	Centralized, distributed, and learning-based MPC and RL.[16]	71% of studies have unreliable protocols.[16] Reliable trials show duration-weighted savings of 16% (residential) and 13% (commercial).[16]
Small HVAC Control Demonstrations in Larger Buildings Often Overestimate Savings [20]	Khabbazi & Kircher, 2026	Multi-zone commercial buildings, analytical model.[20]	Subset-zone optimization vs. full-facility baseline.[20]	Partial-building trials overestimate savings by ignoring cross-zone heat transfer; proposes a boundary-corrected estimation method.[20]

The Intersection of Model Predictive Control and Machine Learning

The integration of Machine Learning (ML) with Model Predictive Control (ML-MPC) addresses the primary limitations of both paradigms, combining the rigorous safety, stability, and constraint guarantees of MPC with the high-dimensional representation and adaptive capabilities of ML.[23] This intersection is categorized into three core methodologies.[25]

Model Learning and Surrogate Modeling

The traditional bottleneck of MPC is the substantial engineering effort required to construct accurate physical models (white-box models) or state-space models via classical system identification.[8] ML-MPC addresses this by using data-driven regression algorithms to learn the

system's dynamic transition function from historical telemetry data.[23]

- **Physics-Informed Neural Networks (PINNs):** PINNs act as high-fidelity digital twins of physical components.[26] For example, in large-scale cooling infrastructure like the CERN accelerator HVAC systems, the plant is modeled by breaking down the system topology into interconnected physical component blocks (mixing chambers, cooling coils, heaters, fans, and dynamic thermal zones).[26] Each block is modeled using a neural network, trained on telemetry data, and constrained by physical laws (such as mass conservation and thermodynamics) to ensure physical plausibility and bounded predictions.[26]
- **NARX Metamodels:** Nonlinear Autoregressive Exogenous (NARX) models represent dynamic systems where the future output depends on both historical inputs and historical outputs.[7] Generated from high-fidelity 1D physics-based simulation platforms (like GT-SUITE), these surrogate models can capture highly nonlinear transient relationships (such as chiller valve positions, fan speeds, and multi-point temperatures).[7] These metamodels achieve prediction accuracies within 5% of full physics simulations while running several orders of magnitude faster, enabling real-time NMPC optimization.[7]
- **Grey-Box RC Networks:** Grey-box models combine physical knowledge with data-driven regression.[13] In thermal environments, buildings and server racks are modeled as equivalent electrical Resistance-Capacitance (RC) networks, where temperatures represent node voltages and heat flows represent currents.[13] The dynamic temperature equation at any node i is formulated as:

$$\dot{T}_i = \sum_{j} \frac{T_j - T_i}{C_i R_{ij}} + \sum_{k} \frac{\dot{Q}_k}{C_i}$$
Here, C_i represents the thermal capacitance of the node, R_{ij} represents the thermal resistance between adjacent nodes i and j, and $\dot{Q}_k$ represents external heat disturbances or direct cooling inputs.[13] Telemetry data is utilized to identify these lumped parameters (R and C), after which the system is automatically converted into a linear state-space formulation suitable for rapid convex optimization.[13]

- **Data-Enabled Predictive Control (DeePC):** An alternative data-driven predictive method is Data-Enabled Predictive Control (DeePC), which bypasses explicit model identification entirely.[19] Based on Willems' Fundamental Lemma of behavioral systems theory, DeePC performs predictive control by using raw historical input-output trajectories directly in the optimization problem to represent the system's dynamic behavior, making it highly effective for complex, non-stationary systems where mathematical modeling is difficult.[19]

Controller Parameterization Tuning

Rather than modeling the physical plant, ML is also used to optimize the parameterization of the MPC controller itself.[25] Designing MPC weights (Q and R matrices) to satisfy multi-objective criteria is traditionally a manual, trial-and-error process.[1] Bayesian Optimization and other regression techniques are deployed to automatically tune these weights and safety margin parameters based on closed-loop performance metrics, safely finding optimal tuning

parameters without human intervention.[25]

MPC as a Safety Filter for Machine Learning

In this framework, a model-free machine learning agent (such as a deep RL algorithm) computes control actions optimized for complex, long-term economic objectives, but those actions are passed through an MPC safety filter.[25] The MPC safety filter solves a localized optimization problem to verify if the RL-proposed actions will cause constraint violations over the prediction horizon.[25] If a violation is predicted, the MPC filter projects the action back onto the boundary of the safe admissible control set $\mathcal{U}$, enabling safe exploration during physical training.[25]

- **Differentiable MPC (e.g., Gnu-RL):** Differentiable MPC bridges the gap between optimization and learning-based policies.[28] In frameworks like Gnu-RL, the control policy is parameterized as a differentiable MPC controller, allowing the system to backpropagate gradients from the control loss directly through the optimization solver and dynamic transition equations.[28] This integrates the planning structure of MPC with the end-to-end learning capabilities of reinforcement learning.[28]

Neural Network Approximation of MPC (NN-MPC)

While NMPC provides optimal control, its real-time industrial deployment is often restricted by the online computational burden of solving non-convex optimization problems at high sampling frequencies.[15] Neural Network MPC (NN-MPC) solves this problem by shifting the computational burden offline.[15]

During the offline phase, a traditional solver (such as CasADi or IPOPT) is used to solve thousands of OCP instances across a grid of initial states (x) and operating conditions (p, such as varying thermal loads and weather forecasts).[2] The corresponding optimal control inputs (u^*) are recorded, creating a training dataset of input-output pairs.[15] A deep feedforward neural network is then trained to approximate this mapping:

$$\hat{u} = \mathcal{NN}(x, p)$$

The network is optimized using algorithms like Adam to minimize the Mean Squared Error (MSE) between the network's predictions and the actual optimal actions.[15]

Once trained, the neural network is deployed directly onto real-world control hardware, replacing the iterative numerical solver with simple, deterministic matrix-vector multiplications.[15] Performance benchmarks demonstrate the massive computational advantages of this approximation:

- **Simulation Benchmark:** In dynamic robotic control, a traditional online MPC requiring **40 ms** per iteration was approximated by an NN-MPC requiring only **0.5 ms** (a 100x speedup).[15]
- **Hardware Benchmark:** When compiled onto a real-time industrial controller, the execution time was reduced to **8 microseconds**, representing a speedup factor of approximately **5,000x**.[15]

This massive speedup enables the implementation of advanced predictive control on physical systems with highly nonlinear, high-frequency dynamics that were previously incompatible with traditional optimization software.[15]

Open-Source Software Ecosystem

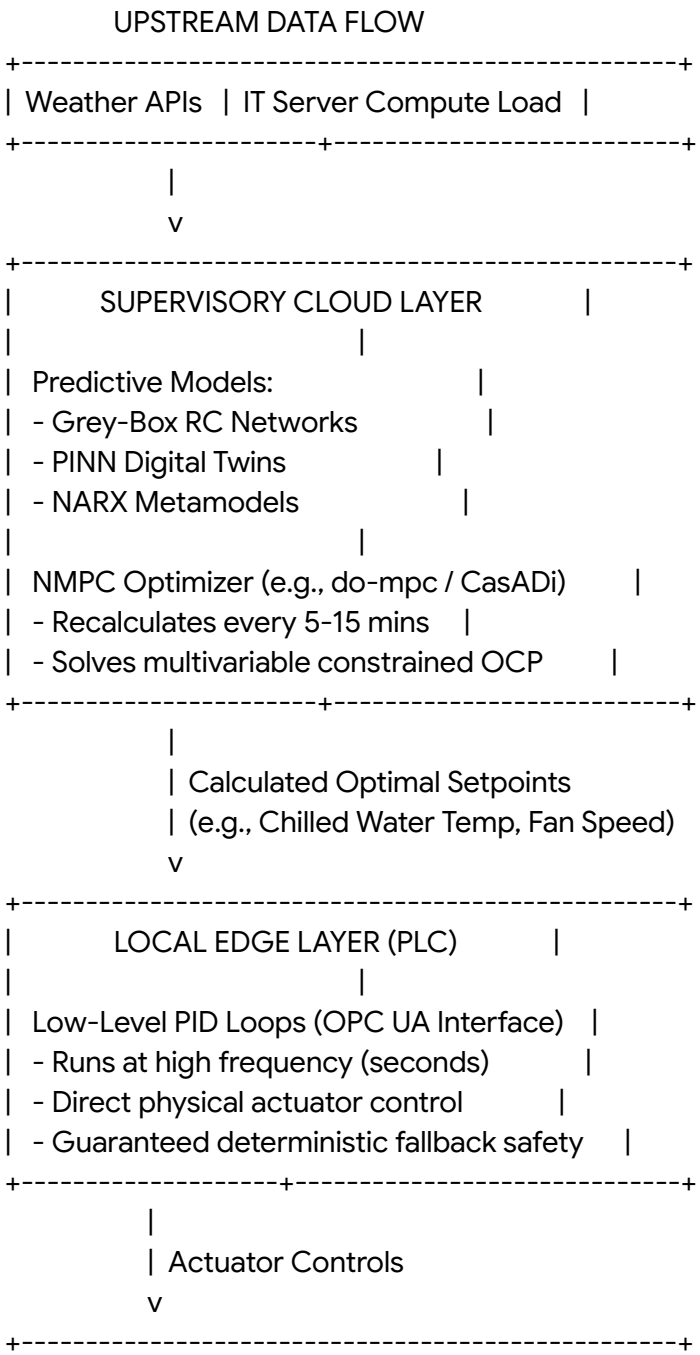
Several packages are available to support the implementation of these hybrid learning and predictive control workflows:

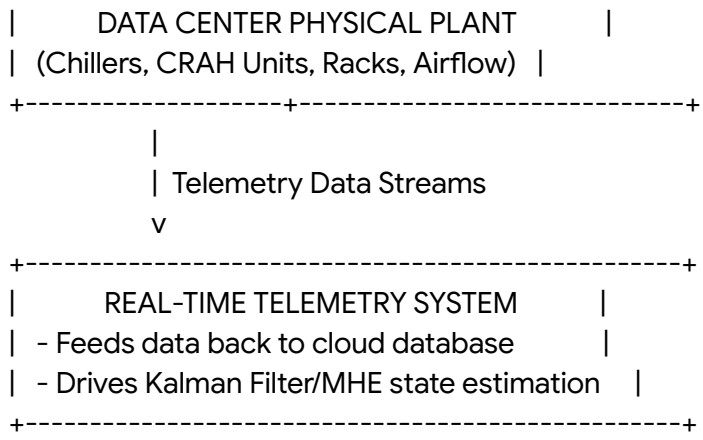
Package Name	Principal Language	Primary Application and Architectural Role	Notable Technical Features
do-mpc [29]	Python	Robust, nonlinear, and economic model predictive control and estimation.[5]	Scenario-tree robust NMPC, Moving Horizon Estimation (MHE), CasADi and IPOPT integration.[5]
FastSim [13]	Python	Automated setup, evaluation, and tuning of MPC controllers for building energy systems.[13]	Automatic state-space derivation, grey-box parameter estimation, time-varying Kalman filtering.[13]
phyvac [30]	Python	Simulation and dataset generation for flexible building HVAC systems.[30]	Generates realistic, fault-injected baseline datasets for training dynamic surrogates.[30]
nMPyC [31]	Python	Solving nonlinear finite-horizon OCPs via algorithmic differentiation.[31]	Black-box optimization interface utilizing CasADi, SciPy, and OSQP solvers.[31]

Engineering Roadmap: Translating an RL-Based

Cooling System to MPC

Transitioning an existing telemetry-driven RL cooling system to a Model Predictive Control architecture involves a structured, hierarchical approach.[7] This transition leverages the existing telemetry pipeline to build predictive capacity while enforcing deterministic physical safety.[7]





Phase I: Telemetry Aggregation and Grey-Box Model Identification

The existing RL infrastructure already captures high-frequency telemetry data, which can be utilized to identify the system's thermal dynamics.[8]

- **Input-Output Mapping:** Collect time-series data of IT server compute loads (representing thermal disturbances), computer room air handler (CRAH) fan speeds, valve positions, ambient weather conditions, and spatial temperature measurements.[7]
- **Parameter Estimation:** Utilize the FastSim library to construct a multi-node thermal RC network of the data center.[13] Automatically solve the parameter estimation problem to find the optimal resistance (R) and capacitance (C) values that minimize the error between the model's predictions and the historical telemetry data.[13]
- **State Space Conversion:** Convert the calibrated RC network into a discrete-time state-space representation [13]:

$$x_{k+1} = A x_k + B u_k + B_d d_k$$
Here, x_k represents spatial room temperatures (states), u_k represents actuator commands (control inputs), and d_k represents IT server thermal loads and ambient weather conditions (disturbances).[5]

Phase II: Dynamic Surrogate Model Training

For complex data centers with highly nonlinear cooling dynamics (such as variable refrigerant flow loops or liquid cooling blocks), a linear model may be insufficient.[7]

- **Data Generation:** Utilize a physical 1D simulation tool, such as GT-SUITE, to build a detailed model of the cooling infrastructure.[7] Run automated sweeps using the Machine Learning Assistant to generate a dataset of thermal behavior across various operating conditions.[7]
- **NARX Training:** Train a Nonlinear Autoregressive Exogenous (NARX) metamodel using deep neural networks to approximate the high-fidelity simulation.[7]
- **Compilation and Export:** Export the trained NARX model as compiled C code or a C MEX block to enable computationally efficient, sub-millisecond evaluation within the

real-time MPC optimizer.[7]

Phase III: Receding Horizon Optimization Configuration

Establish the core components of the MPC solver using the do-mpc Python package.[29]

- **Objective Function Formulation:** Define an economic objective function designed to minimize power usage effectiveness (PUE) [12] and reduce electricity costs.[27] The optimization target is formulated to penalize total electrical power consumption (from chillers, pumps, and fans) while maintaining the spatial temperatures within a target range.[12]
- **Enforcing Constraints:** Define hard boundaries for maximum server inlet temperatures, actuator rate limits to prevent equipment wear, and physical constraints to prevent simultaneous heating and cooling.[7]
- **Predictive Horizon Tuning:** Configure the prediction horizon K to align with the system's thermal inertia.[2] For large data centers, a prediction horizon of 1 to 6 hours with a sampling interval of 5 to 15 minutes is recommended to allow the controller to proactively pre-cool the facility before peak IT workloads or ambient temperature spikes.[12]

Phase IV: Parallel Deployment and Safing

Deploy the MPC using a phased approach to ensure operational safety and build performance trust.[12]

STAGE 1: Recommendation Mode
Legacy/RL Controller => In Control
MPC Solver => Active (Outputs recommended actions to logs for validation)

STAGE 2: MPC Safety Filter
RL Controller => Active (Computes economic actions)
MPC Solver => Active (Intercepts and projects actions onto safe boundary)

STAGE 3: Full Autonomous MPC
MPC Solver => In Control (Sends setpoints to low-level PID loops via OPC UA)

- **Step 1: Recommendation Mode (Shadow Run):** Run the MPC in shadow mode.[12] The controller receives real-time telemetry and calculates optimal setpoints, but these actions are logged without being applied to the physical plant.[7] This allows the engineering team to validate the predictive model's accuracy and optimizer stability.[7]
- **Step 2: Safety Filtering:** Integrate the MPC as an online safety filter downstream of the existing RL policy.[25] This allows the RL agent to handle economic optimization while the

MPC guarantees that the proposed actions respect all safety and operational constraints.[25]

- **Step 3: Direct Supervisory Control:** Transition the system to direct supervisory control.[26] The MPC executes periodically (e.g., every 15 minutes) on a cloud platform, such as OpenShift, and sends the calculated optimal setpoints down to the physical PLCs via an OPC UA server, which are then tracked by high-frequency, edge-level PID loops.[26] This hierarchical architecture combines the predictive capabilities of advanced optimization with the robust, deterministic feedback safety of classical control.[26]

Works cited

1. Predictive Control: Fundamentals and Developments | Wiley, accessed on June 13, 2026, https://www.wiley.com/en-us/Predictive+Control%3A+Fundamentals+and+Developments-p-9781119119548
2. Model Predictive Control Basics | Towards Data Science, accessed on June 13, 2026, https://towardsdatascience.com/model-predictive-control-basics/
3. Model Predictive Control System Design and Implementation Using MATLAB® - ResearchGate, accessed on June 13, 2026, https://www.researchgate.net/profile/Mohamed-Mourad-Lafifi/post/How_to_design_model_predictive_controller_in_a_factory/attachment/5d2dae0b3843b0b9825ae2b9/AS%3A781245130735617%401563274762980/download/Model+Predictive+Control+System+Design+and+Implementation+Using+MATLAB_Wang.pdf
4. MPC lab - MPC Book - Google Sites, accessed on June 13, 2026, https://sites.google.com/berkeley.edu/mpc-lab/mpc-course-material
5. Basics of model predictive control — do-mpc 5.1.1 documentation, accessed on June 13, 2026, https://www.do-mpc.com/en/latest/theory_mpc.html
6. Get Started with Model Predictive Control Toolbox - MathWorks, accessed on June 13, 2026, https://www.mathworks.com/help/mpc/getting-started-with-model-predictive-control-toolbox.html
7. Transforming Data Center Cooling with Digital Twins and MPC, accessed on June 13, 2026, https://www.gtisoft.com/blog-post/transforming-data-center-cooling-from-physics-based-simulation-to-ai-powered-control/
8. MPC vs reinforcement learning for process control - PatSnap, accessed on June 13, 2026, https://www.patsnap.com/resources/blog/articles/mpc-vs-reinforcement-learning-for-process-control/
9. Comparative Field Deployment of Reinforcement Learning and Model Predictive Control for Residential HVAC - arXiv, accessed on June 13, 2026, https://arxiv.org/html/2510.01475v1
10. Reinforcement learning vs Model predictive control | by Abebe Sendek - Medium, accessed on June 13, 2026,

https://airob.medium.com/reinforcement-learning-vs-model-predictive-control-f43f97a0be27
11. MPC for HVAC Energy Efficiency — PatSnap Eureka | PatSnap, accessed on June 13, 2026, https://www.patsnap.com/resources/blog/rd-blog/mpc-for-hvac-energy-efficiency-patsnap-eureka/
12. AI-Driven Predictive Control for Data Center HVAC Systems - Heat Pumping Technologies, accessed on June 13, 2026, https://heatpumpingtechnologies.org/articles/heat-pumping-technologies-magazine-vol-43-no-3-2025/ai-driven-predictive-control-for-data-center-hvac-systems/
13. A Python-Based Toolbox for Model Predictive ... - Purdue e-Pubs, accessed on June 13, 2026, https://docs.lib.purdue.edu/cgi/viewcontent.cgi?article=1281&context=ihpbc
14. Intelligent HVAC Control: Comparative Simulation of Reinforcement Learning and PID Strategies for Energy Efficiency and Comfort Optimization - MDPI, accessed on June 13, 2026, https://www.mdpi.com/2227-7390/13/14/2311
15. From MPC to NN-MPC for real-time mechatronic control, accessed on June 13, 2026, https://nobleo-technology.nl/neural-network-model-predictive-control-nn-mpc/
16. Lessons learned from field demonstrations of model predictive control and reinforcement learning for residential and commercial HVAC: A review - arXiv, accessed on June 13, 2026, https://arxiv.org/html/2503.05022v1
17. Lessons learned from field demonstrations of model predictive control and reinforcement learning for residential and commercial HVAC: A review | Request PDF - ResearchGate, accessed on June 13, 2026, https://www.researchgate.net/publication/394356423_Lessons_learned_from_field_demonstrations_of_model_predictive_control_and_reinforcement_learning_for_residential_and_commercial_HVAC_A_review
18. RL vs MPC for Residential HVAC Control | PDF | Mathematical Optimization - Scribd, accessed on June 13, 2026, https://www.scribd.com/document/931063892/Comparative-Field-Deployment-of-Reinforcement-Learning-and-Model-Predictive-Control-for-Residential-HVAC
19. Lessons learned from field demonstrations of model predictive control and reinforcement learning for residential and commercial - arXiv, accessed on June 13, 2026, https://arxiv.org/pdf/2503.05022
20. Arash Jalil Khabbazi, accessed on June 13, 2026, https://ajkhabbazi.github.io/
21. How MPC is Transforming HVAC Systems for a Greener Future - PartsHnC, accessed on June 13, 2026, https://www.partshnc.com/blog/hvac-optimization-sustainable-mpc-next-gen-technology
22. [2503.05022] Lessons learned from field demonstrations of model predictive control and reinforcement learning for residential and commercial HVAC: A review - arXiv, accessed on June 13, 2026, https://arxiv.org/abs/2503.05022
23. A Survey on Learning-Based Model Predictive Control: Toward Path Tracking

Control of Mobile Platforms - ResearchGate, accessed on June 13, 2026, https://www.researchgate.net/publication/358621004_A_Survey_on_Learning-Based_Model_Predictive_Control_Toward_Path_Tracking_Control_of_Mobile_Platforms
24. A Comparison of Model-Based MPC and Machine Learning MPC | by Abebe Sendek, accessed on June 13, 2026, https://airob.medium.com/a-comparison-of-model-based-mpc-and-machine-learning-mpc-9e051d689008
25. Learning-Based Model Predictive Control: Toward Safe Learning in ..., accessed on June 13, 2026, https://www.annualreviews.org/eprint/2STMCYXGPHBRMTDP9W2D/full/10.1146/annurev-control-090419-075625
26. Neural network-based model predictive control for energy efficient ..., accessed on June 13, 2026, https://proceedings.jacow.org/icalepcs2025/pdf/MOCG002.pdf
27. AI Chiller Plant Optimization: From Model Predictive Control to Deep Reinforcement Learning | CKY HVAC Engineering, accessed on June 13, 2026, https://www.cky.com.tw/en/insights/ai-hvac-optimization
28. Gnu-RL: A Practical and Scalable Reinforcement ... - Frontiers, accessed on June 13, 2026, https://www.frontiersin.org/journals/built-environment/articles/10.3389/fbuil.2020.562239/full
29. Model predictive control python toolbox — do-mpc 5.1.1 documentation, accessed on June 13, 2026, https://www.do-mpc.com/
30. Phyvac: a Python module for highly flexible HVAC system simulation, and fault dataset generation as an application example - IBPSA Publications, accessed on June 13, 2026, https://publications.ibpsa.org/conference/paper/?id=bs2023_1370
31. nmpyc - PyPI, accessed on June 13, 2026, https://pypi.org/project/nmpyc/
32. A sample code for the development of Model Predictive Control in Buildings - GitHub, accessed on June 13, 2026, https://github.com/robgaray/Model-Predictive-Control-in-Buildings